

L10 — Data Grouping, Dataset Merging, and JSON Reading (Combined)

Exported from the provided Jupyter notebook (L10_combined.ipynb).

****Combining DataFrames****

Code

```
import numpy as np
import pandas as pd
`pd.concat()`
```

`pd.concat()` is a **Pandas** function used to combine multiple DataFrames or Series along a specified axis.

- **`axis=0`** → Stacks data **vertically** (adds rows).
- **`axis=1`** → Combines data **horizontally** (adds columns).
- **`join='outer'`** → Keeps all indexes (union of indexes).
- **`join='inner'`** → Keeps only common indexes (intersection of indexes).

It is commonly used to append datasets or combine related data structures into a single DataFrame.

Code

```
df1=pd.DataFrame(np.arange(9).reshape(3,3), columns=['a','b','c'], index=['one',
'two', 'three'])
df1
```

Code

```
df2=pd.DataFrame(np.arange(6).reshape(-1,2), columns=['d','e'], index=['three', 'two',
'One'])
df2
```

Code

```
nums = [1, 2, 3, 4, 5]
print(nums[-3::-1])
print(nums[-3:-1])
```

More Tricky List Slicing Examples

- **`nums = [1, 2, 3, 4, 5]`**
- **`nums[::2]`** → Every other element: **`[1, 3, 5]`**
- **`nums[::-1]`** → Reversed list: **`[5, 4, 3, 2, 1]`**
- **`nums[1:-1]`** → All except first and last: **`[2, 3, 4]`**
- **`nums[-4:-1]`** → From 2nd to 4th from end: **`[2, 3, 4]`**
- **`nums[1:100]`** → Slices safely even if stop > length: **`[2, 3, 4, 5]`**
- **`nums[-100:3]`** → Slices safely even if start < 0: **`[1, 2, 3]`**
- **`nums[3:1:-1]`** → Slices backwards: **`[4, 3]`**
- **`nums[1:1]`** → Empty list: **`[]`**

Code

```
t = (1,2,3,4,5)
print(t[1:-1])
```

Code

```
df1=pd.DataFrame(np.arange(9).reshape(3,3), columns=['a','b','c'], index=['one',
'two', 'three'])
df1
```

Code

```
df2=pd.DataFrame(np.arange(6).reshape(-1,2), columns=['d','e'], index=['three', 'two',
'one'])
df2
```

Code

```
pd.concat([df1, df2], axis=1)
```

Code

```
pd.concat([df1, df2], axis=0)
```

Code

```
nums = [1,2,3,4,5]
print(nums[:99:])
dict = {'a': 1, 'b': 2, 'c': 3}
print(set(dict.keys()))
```

Code

```
df = pd.concat([df1, df2], axis=0) # Stack rows vertically (default
behavior), matching columns by name
print(df)
print(df.iloc[0:3])# First 3 rows from df1 (since df1 is on top) and all columns
(including the new ones from df2)
print(df.iloc[0:3, 0:2]) # First 3 rows from df1 and first 2 columns (a and b)
print(df.iloc[1::, 0:-1]) # All rows from index 1 onward and all columns from right
to left
print(df.iloc[1::, 0:]) # All rows from index 1 onward and all columns (including the
new ones from df2)
```

Code

```
pd.concat([df1, df2], axis=1) # Combine columns side-by-side, aligning
rows by index (outer join by default)
```

Code

```
pd.concat([df1, df2], axis=1, join='outer') # Side-by-side with all indexes included
(union of indexes, may create NaNs)
```

Code

```
pd.concat([df1, df2], axis=1, join='inner') # Side-by-side with only common indexes
kept (intersection of indexes)
```

Code

```
pd.concat([df1, df2], axis=1, join='inner')
```

`pd.merge()`

`pd.merge()` is a **Pandas** function used to combine DataFrames based on common column(s) or key(s) (similar to SQL joins).

- **`on='col'`** → Column name used as the join key (must exist in both DataFrames).
- **`left_on` / `right_on`** → Use different column names as join keys.
- **`how='inner'`** → Keeps only matching rows (default).
- **`how='left'`** → Keeps all rows from the left DataFrame.
- **`how='right'`** → Keeps all rows from the right DataFrame.
- **`how='outer'`** → Keeps all rows from both DataFrames.

It is commonly used when combining datasets that share a common identifier (e.g., ID, email, product code).

Code

```
df3=pd.DataFrame([[ 'a', 'b', 'c'], [ 'd', 'e', 'f'], [ 'g', 'h', 'i']], columns=[ 'col1',  
'col2', 'col3'])  
df3
```

Code

```
df4=pd.DataFrame([[ 'x', 1, 'i'], [ 'e', 2, 'f'], [ 'b', 3, 'e'], [ 'z', 4, 'h']], columns=[ 'col2',  
'col4', 'col5'])  
df4
```

Code

```
pd.merge(df3, df4, left_on='col2', right_on='col5', how='inner') # Match df3['col2']  
with df4['col5'], keep only matching rows
```

Code

```
pd.merge(df3, df4, how='outer') # Keep all rows from both DataFrames (union  
of keys, fills unmatched with NaN)
```

Code

```
pd.merge(df3, df4, how='inner') # Keep only rows with matching keys in both  
DataFrames (intersection)
```

Code

```
pd.merge(df3, df4, on='col2', how='left') # Keep all rows from df3, match on 'col2',  
fill unmatched df4 values with NaN
```

Code

```
pd.merge(df3, df4, left_on='col2', right_on='col5', how='inner') # Match df3['col2']  
with df4['col5'], keep only matching rows
```

Code

```
pd.merge(df3, df4, on='col2', how='right') # Keep all rows from df4, match on 'col2',  
fill unmatched df3 values with NaN
```

Code

```
df1 = pd.DataFrame({'key': [1, 2], 'A': ['x', 'y']})  
df2 = pd.DataFrame({'key': [2, 3], 'B': ['w', 'z']})  
res = pd.merge(df1, df2, on='key', how='outer')  
res
```

Code

```
df1
```

Code

```
df2
```

Code

```
res
```

Code

```
res_concat_like_merge = (  
    pd.concat(  
        [df1.set_index("key"), df2.set_index("key")],  
        axis=1,  
        join="outer"  
    )  
    .reset_index()  
)  
res_concat_like_merge
```

Code

```
matrix = [[1, 2], [3, 4]]  
print([num * 2 for row in matrix for num in row if num % 2 == 0])  
  
arr1 = np.array([[1], [2], [3]])  
arr2 = np.array([10, 20])  
print((arr1 + arr2))
```

Code

```

res_concat_like_merge = (
    pd.concat(
        [df1.set_index("key"), df2.set_index("key")],
        axis=0,
        join="outer"
    )
    .reset_index()
)
res_concat_like_merge

```

Code

```

import pandas as pd
df1 = pd.DataFrame({'id': [1, 2], 'val1': ['A', 'B']})
df2 = pd.DataFrame({'id': [2, 3], 'val2': ['C', 'D']})
res = pd.merge(df1, df2, on='id', how='left')
res

```

Code

```

import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3]})
df['A'] = df['A'].apply(lambda x: x**2)
print(df['A'].tolist())

```

Code

```

df = pd.DataFrame({"A": [1,2,3]})
df["A"] = df["A"].apply(lambda k: k**2)
print(df)

```

Code

```

arr = np.array([1, 2, 4, 4, 5])
print(np.where(arr > 3, True, False))
print(arr[arr > 1])

```

Code

```

import pandas as pd
import numpy as np
df = pd.DataFrame({'A': [1, np.nan, np.nan],
                    'B': [4, np.nan, 6],
                    'C': [7, np.nan, 9]})
print(df.dropna(how='all', axis=1))
print(df.dropna(how='all', axis=0))

print(df.iloc[0:3, 0:2])

```

Code

```
import pandas as pd
df1 = pd.DataFrame({'id': [1, 2], 'val1': ['A', 'B']})
df2 = pd.DataFrame({'id': [2, 3], 'val2': ['C', 'D']})
res = pd.merge(df1, df2, on='id', how='outer')
res
```

Code

```
import pandas as pd

data = {
    'Dept': ['IT', 'IT', 'HR', 'HR', 'IT', 'HR'],
    'Year': [2023, 2024, 2023, 2024, 2023, 2023],
    'Sales': [100, 150, 200, 250, 50, 50]
}
df = pd.DataFrame(data)

print(df)
```

Code

```
import pandas as pd
df1 = pd.DataFrame({'id': [1, 2], 'val1': ['A', 'B']})
df2 = pd.DataFrame({'id': [3, 4], 'val2': ['C', 'D']})
res = pd.merge(df1, df2, on='id', how='inner')
res
```

Code

```
result = df.groupby(['Dept', 'Year'])[['Sales']].sum()
print(result)
```

Code

```
import numpy as np
arr = np.arange(8).reshape(-1, 2)

arr
```

Code

```
arr = np.arange(6).reshape(2, 3)
arr
```

Code

```
print({x: x**2 for x in (1, 2, 3)})
```

Code

```

t = (30, 20)          # Basic tuple with integers
print(t)
t = ("Apple", "Banana") # Tuple of strings
t = (1, "Hello", 3.14) # Mixed data types
t = 1, 2, 3           # "Packing": Parentheses are actually optional
t = (5,)               # Singleton: MUST have a trailing comma for one item
empty = ()             # An empty tuple

```

Code

```

# Goal: Square every number in a list
nums = [1, 2, 3, 4]
squared = list(map(lambda x: x**2, nums))
print(squared) # Output: [1, 4, 9, 16]

# Goal: Convert a list of strings to integers
prices = ["10", "20", "30"]
clean_prices = list(map(int, prices))
print(clean_prices) # Output: [10, 20, 30]

# Goal: Capitalize names in a list
names = ["ALice", "bob", "charlie"]
caps = list(map(str.upper, names))
# Output: ['ALICE', 'BOB', 'CHARLIE']
print(caps)

list(zip([1, 2], ['a', 'b'])) # Output: [(1, 'a'), (2, 'b')]

```

Code

```

import pandas as pd
df1 = pd.DataFrame({'id': [1, 2], 'val1': ['A', 'B']})
df2 = pd.DataFrame({'id': [2, 3, 4], 'val2': ['C', 'D', 'E']})
res = pd.merge(df1, df2, on='id', how='right')
res

```

Code

```

import pandas as pd
import numpy as np
df = pd.DataFrame({'A': [1, np.nan, np.nan],
                   'B': [4, np.nan, 6],
                   'C': [7, np.nan, 9]})
print(df.dropna(how='all').shape)
print(df.dropna(how='all'))
df = pd.DataFrame({'A': [1, np.nan, np.nan], #these are columns
                   'B': [4, np.nan, 6],
                   'C': [7, np.nan, 9]})
df

```

Code

```
keys = ['a', 'b']
vals = [1, 2]
print(list(zip(keys,vals)))
```

Code

```
s1 = {1, 2, 3}
s2 = {1, 2, 3}
print(s1.issubset(s2))
```

Code

```
import pandas as pd
df1 = pd.DataFrame({'A': [1, 2, 3]})
df2 = pd.DataFrame({'B': [4, 5, 6]})
res = pd.concat([df1, df2], axis=0)
res
```

Code

```
arr = np.array([[3, 8], [9, 2]])
print(arr.max(axis=1)) # across the columns
```

Code

```
# CAREFUL
result = [x * 2 for x in range(5) if x % 2 == 0]
print(result)
```

Code

```
import pandas as pd
data = {'city': ['NYC', 'LA', 'NYC', 'LA', 'NYC'],
        'sales': [100, 200, 150, 300, 50]}
df = pd.DataFrame(data)
print(df.groupby('city')['sales'].sum())
```

Code

```
cond = np.array([True, False, True])
print(np.where(cond, [1, 2, 3], [4, 5, 6]))
```

Code


```
d = {'x': [10, 20], 'y': [30, 40], 'z': [50, 60]}
import pandas as pd
df = pd.DataFrame(d)
print(df.shape)
```

Code

```
lst = [5, 10, 15, 20, 25]
lst.extend([10,20])
print(lst.count(10))
print(lst)
```

Code

```
lst = [5, 10, 15, 20, 25]
lst.append([10,20])
print(lst.count(10))
print(lst)
```

Code

```
import pandas as pd
df1 = pd.DataFrame({'id': [1, 2, 2], 'val': ['A', 'B', 'C']})
df2 = pd.DataFrame({'id': [2, 3], 'score': [100, 200]})
res = pd.merge(df1, df2, on='id', how='inner')
print(res)
print(df1)
print(df2)
```

Code

```
df1 = pd.DataFrame({'id': [1, 1], 'val': ['A', 'B']})
df2 = pd.DataFrame({'id': [1, 1], 'score': [10, 20]})
res = pd.merge(df1, df2, on='id', how='inner')
print((res.shape))
print(res)
```

Code

```
d = {"a": 100, "b": 200}
print("a" in d.keys())
print(d["a"])
print(d.get("a"))
print(d.get("c", None))
```

JSON Reading + Type Check (Added 2026-03-05 18:13:07)

This section combines:

- Reading JSON using Python `json.load` / `json.loads`
- Reading JSON into pandas with `pd.read\_json` (including JSONL)
- Determining whether loaded JSON is a `dict` or `list`

Code

```
import json
import pandas as pd

# -----
# A) Read JSON from a file -> Python object (dict or list)
# -----
# data.json can be either:
# { ... } -> dict
# [ ... ] -> list
with open("data.json", "r", encoding="utf-8") as f:
    data = json.load(f)

print("Python type from json.load:", type(data))

if isinstance(data, dict):
    print("This JSON root is a dictionary (starts with { ... }).")
elif isinstance(data, list):
    print("This JSON root is a list (starts with [ ... ]).")
else:
    print("Root type is neither dict nor list (rare).")

# -----
# B) Read JSON from a string -> Python object
# -----
s = '{"name": "Mike", "age": 30}'
obj = json.loads(s)
print("\nFrom json.loads:", obj, "| type:", type(obj))

# -----
# C) Read JSON directly into pandas DataFrame
# -----
# 1) Standard "records" JSON (array of objects)
# employees.json example:
# [
#   {"id": 1, "name": "Mike"},
#   {"id": 2, "name": "Sara"}
# ]
# df = pd.read_json("employees.json")

# 2) Line-delimited JSON (JSONL / NDJSON) where each line is a JSON object
# events.jsonl example:
# {"event": "login", "user": "Mike"}
# {"event": "logout", "user": "Sara"}
# df = pd.read_json("events.jsonl", lines=True)

# 3) Flatten nested JSON (if needed)
# with open("nested.json", "r", encoding="utf-8") as f:
#     nested = json.load(f)
# df = pd.json_normalize(nested, sep=".")
```